SOFTWARE REQUIREMENTS SPECIFICATION

Voice Notes Management System (NWVS)

A Note-Taking Application with Multilingual Voice Transcription and Real-Time Translation

2411CS020571-Surya

2411CS020624-Rohith

2411CS020548-Rudraksh

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements of the Voice Notes Management System (internally referred to by its codebase name, NWVS). The document is intended for developers, testers, project reviewers, and any stakeholder who needs a precise, implementation-grounded reference for what the system does and how it is built. This SRS was reverse-engineered directly from the delivered source code (Spring Boot backend and React frontend) so that it reflects the system as actually implemented, rather than a system as originally envisioned.

1.2 Scope

The Voice Notes Management System is a full-stack web application that allows an individual user to capture, organize, and manage personal notes with support for voice-based dictation and cross-language translation. The system consists of:

- A Java/Spring Boot REST API backend responsible for authentication, note management, dashboard analytics, and text translation.
- A React (Vite) single-page application frontend that consumes the REST API and provides browser-based voice transcription via the Web Speech API.
- A relational database (MySQL for production use, H2 available for local/embedded use) that persists users, notes, OTP records, and password-recovery requests.

The system supports account registration with email/SMS one-time-password (OTP) verification, two-factor login, a token-based “confirm by email link + OTP” password recovery flow, full CRUD note management with pinning/archiving/favoriting, category-based organization, keyword search, pagination and sorting, a usage dashboard, browser-based speech-to-text dictation in English, Hindi, and Telugu, and on-demand translation of note text between those three languages.

Out of scope: native mobile applications, offline-first data synchronization, multi-user collaboration on shared notes, file/image attachments to notes, and payment or subscription functionality. None of these exist in the current codebase.

1.3 Intended Audience

- Software developers extending or maintaining the codebase
- QA engineers designing test plans and test cases
- Technical reviewers and academic evaluators assessing the project
- DevOps personnel responsible for deployment and configuration

1.4 Definitions, Acronyms, and Abbreviations

Term	Definition
SRS	Software Requirements Specification

Term	Definition
API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token – used for stateless session authentication
OTP	One-Time Password – a 6-digit code used for account verification and 2FA
2FA	Two-Factor Authentication
CRUD	Create, Read, Update, Delete
DTO	Data Transfer Object
JPA	Jakarta Persistence API (used via Hibernate ORM)
CORS	Cross-Origin Resource Sharing
SPA	Single-Page Application
Web Speech API	Browser-native JavaScript API used for speech recognition (STT) and speech synthesis (TTS)
NWVS	Codebase / package name of the system (org.nwvs); shorthand used for “Notes With Voice System” in this document

1.5 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications
- Project README.md (Voice Notes Management System)
- Spring Boot 4.1 / Spring Security 7 official documentation
- MyMemory Translation API documentation (api.mymemory.translated.net)
- W3C Web Speech API Community Group specification
- Project Postman collection: voice_notes_system.postman_collection.json

1.6 Document Overview

Section 2 provides a high-level description of the product, its users, and the environment it runs in. Section 3 enumerates detailed functional requirements grouped by feature area. Section 4 specifies external interface requirements (UI, API, hardware, software, communications). Section 5 defines non-functional requirements. Section 6 documents the data model. Section 7 describes system architecture and design constraints. Appendices provide the full REST API reference and default/seed data.

2. Overall Description

2.1 Product Perspective

The Voice Notes Management System is a new, self-contained product rather than a component of a larger system. It follows a classic three-tier architecture:

- Presentation tier: a React SPA (built with Vite) served independently and communicating with the backend over HTTPS/REST.
- Application tier: a Spring Boot REST API exposing versionless endpoints under /api, handling authentication, business rules, and orchestration of external services.
- Data tier: a relational database (MySQL in production configuration; H2 is included as an embedded/dev alternative) accessed through Spring Data JPA / Hibernate.

The application also integrates with three external, third-party services: the MyMemory public translation API (for cross-language text translation), an SMTP mail server (for transactional email such as OTPs and password-recovery links), and Twilio (for OTP and recovery delivery via SMS). The browser's built-in Web Speech API is used on the client for speech-to-text; no server-side speech processing exists.

2.2 Product Functions

At a high level, the system provides the following functional groups (each detailed further in Section 3):

- User registration, OTP-based email/SMS verification, and secure login with mandatory two-factor authentication.
- Token-based “forgot password” recovery requiring the user to positively confirm the request via an emailed link before an OTP and reset form are issued.
- Full lifecycle management of notes: create, read, update, delete, pin, archive, and mark as favorite.
- Organization and discovery of notes through category tagging, free-text search, filtering, sorting, and pagination.
- A dashboard summarizing note counts (total, pinned, favorite, archived) and per-category statistics.
- In-browser voice dictation of note content in English, Hindi, or Telugu using the Web Speech API.
- On-demand translation of note text between English, Hindi, and Telugu via a backend translation service with an offline fallback.
- Self-service user profile viewing.

2.3 User Classes and Characteristics

User Class	Description	Technical Expertise
Registered End User	An individual who creates an account, verifies it via OTP, and manages personal voice/text notes.	General computer/smartphone literacy; no technical background required.
Unauthenticated Visitor	A person who has reached the login/registration screen but has not yet created or verified an account.	General computer literacy.
System Administrator /	Person responsible for deploying, configuring	Software engineering /

User Class	Description	Technical Expertise
Developer	(database, SMTP, Twilio, JWT secret), and maintaining the application.	DevOps background.

Note: the current implementation defines a single application-level role for authenticated users; there is no administrative role, permission tier, or multi-tenant concept in the codebase. All authenticated users have identical capabilities, scoped strictly to their own data.

2.4 Operating Environment

2.4.1 Backend

- Java 21 (JDK)
- Spring Boot 4.1.x, Spring Security 7.0, Spring Data JPA, Spring MVC, Spring Mail
- MySQL 8.x (primary datastore); H2 (in-memory/embedded, available for development)
- Runs as a standalone executable JAR (embedded servlet container) listening on port 8080 by default

2.4.2 Frontend

- Node.js 18+ and npm for build tooling
- React 19 with Vite build system and Rolldown bundler
- React Router DOM for client-side routing
- Axios for HTTP communication with the backend API
- Bootstrap 5 and custom CSS for styling; Material icon set
- Runs in any modern desktop browser; voice dictation requires a browser that implements the Web Speech API (e.g., Chrome/Edge). Development server listens on port 5173.

2.4.3 Third-Party Services

- MyMemory Translation API (https://api.mymemory.translated.net) – outbound HTTPS calls from the backend.
- SMTP mail relay (configured for Gmail SMTP by default) – outbound TLS on port 587.
- Twilio SMS API – for OTP and password-recovery SMS delivery.

2.5 Design and Implementation Constraints

- Authentication is stateless; the backend issues JSON Web Tokens and performs no server-side session storage (SessionCreationPolicy.STATELESS).
- CORS is restricted at the framework level to two allowed origins: http://localhost:5173 and http://localhost:3000, reflecting a local-development deployment target as shipped.
- Passwords and OTP codes are never stored in plain text; both are hashed with BCrypt before persistence.
- Voice-to-text conversion is performed entirely client-side using the browser's Web Speech API; the backend never receives or processes raw audio.

- Translation depends on an external, rate-limited public API (MyMemory); the backend includes a deterministic offline fallback for when that service is unreachable or exceeds its quota.
- The object-relational mapping layer is configured with hibernate.ddl-auto=update, meaning schema changes are applied automatically at startup rather than through versioned migration scripts.
- Category values are restricted to a fixed, closed enumeration (Work, Study, Meeting, Ideas, Shopping, Personal, Travel, Others) defined in code; adding a category requires a code change and redeploy.

2.6 Assumptions and Dependencies

- A MySQL server is reachable at the configured host/port before the backend starts (schema is auto-created if absent).
- Valid SMTP credentials and a valid Twilio account are available and network-reachable for OTP/email/SMS features to function; if these are unset or invalid, registration/login OTP delivery and password recovery will fail even though the corresponding REST calls otherwise succeed.
- End users access the system through a browser that supports the Web Speech API for the voice-dictation feature; browsers without this API can still use the application for typed note-taking and translation.
- The MyMemory API's free usage tier is assumed sufficient for the application's translation volume; no API key/authentication is configured for it.

3. System Features and Functional Requirements

Each feature is described with its purpose, the primary flow of events, and enumerated, individually testable functional requirements (FR-x.x), derived directly from controller, service, and DTO implementations in the codebase.

Use Case Diagram

The diagram below summarizes system-level use cases and the actors that participate in them. Two primary (human) actors interact directly with the system: an Unauthenticated Visitor, who can register, log in, and initiate password recovery; and a Registered User, who additionally manages notes, uses voice dictation, requests translations, and views the dashboard and profile. Two secondary (system) actors support the use cases from outside the application boundary: the Email/SMS Gateway (SMTP + Twilio), which delivers OTP codes and recovery links, and the MyMemory Translation API, which performs the actual text translation. The <<include>> relationships show that both Register Account and Login (Two-Factor Authentication) mandatorily incorporate the Verify OTP use case as part of their flow.

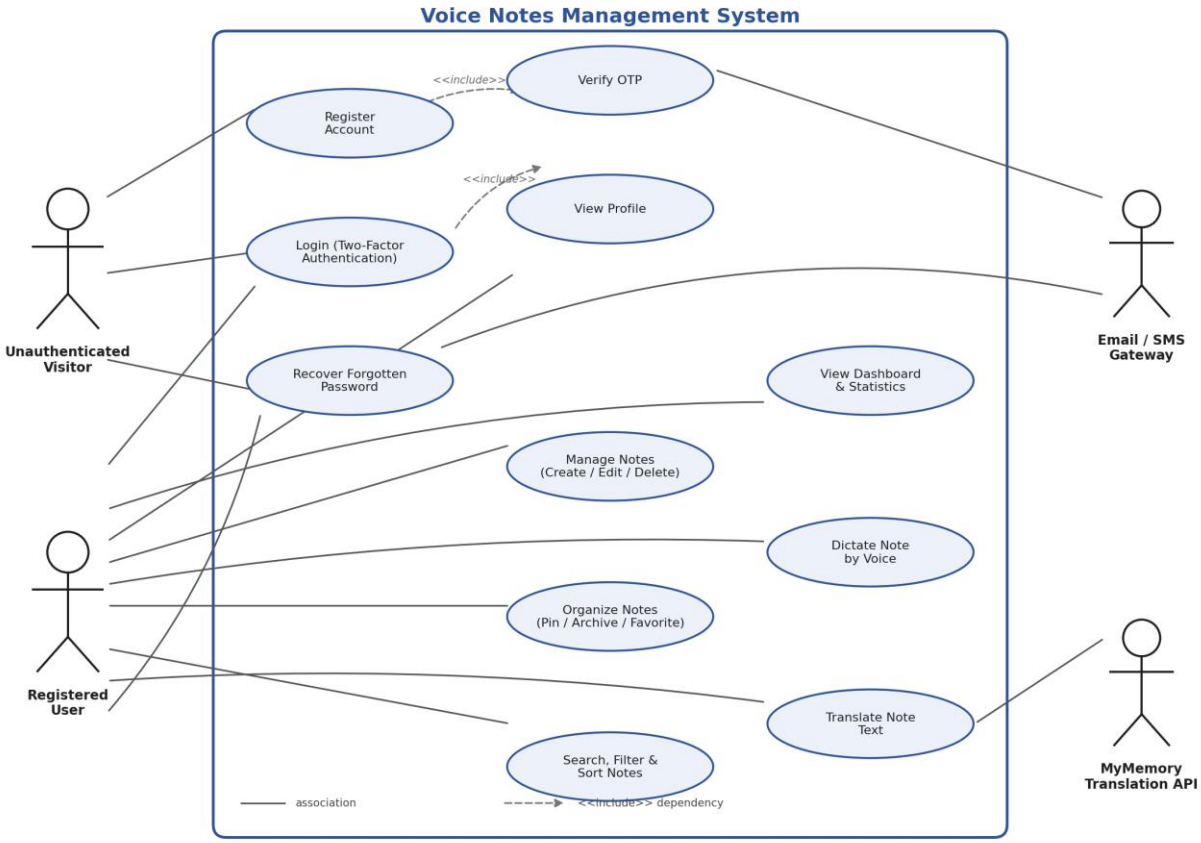

Figure 3.1 – Use Case Diagram: Voice Notes Management System

3.1 User Registration and Account Verification

Description: Allows a new user to create an account. The account is created in an unverified state and cannot log in until the emailed/SMS-delivered OTP is confirmed.

Functional Requirements

- FR-3.1.1: The system shall allow a visitor to submit name, email, phone number, password, and password confirmation to register a new account.
- FR-3.1.2: The system shall reject registration if password and confirmation password do not match.
- FR-3.1.3: The system shall reject registration if the supplied email is already registered to another account.
- FR-3.1.4: The system shall reject registration if the supplied phone number is already registered to another account.
- FR-3.1.5: The system shall validate that the email conforms to a standard email address pattern and the phone number contains 10–15 digits (optionally prefixed with '+').
- FR-3.1.6: The system shall enforce a minimum password length of 8 characters at registration.
- FR-3.1.7: The system shall store the password using a one-way BCrypt hash; plain-text passwords shall never be persisted.

- FR-3.1.8: Upon successful registration, the system shall create the user record with verified = false and automatically generate and dispatch a 6-digit OTP (purpose = REGISTER) to the user's email and phone number.
- FR-3.1.9: The system shall provide an endpoint to submit the registration OTP; upon a correct, unexpired, unused OTP, the system shall set the account's verified flag to true.
- FR-3.1.10: An unverified account shall not be permitted to complete login (see Section 3.2).

3.2 Authentication and Two-Factor Login

Description: Login is a two-step process. Step 1 validates credentials and issues a one-time login OTP. Step 2 verifies that OTP and issues a JWT access token.

Functional Requirements

- FR-3.2.1: The system shall accept an email-or-phone identifier and password for login (step 1).
- FR-3.2.2: The system shall reject login with a generic “invalid credentials” message if the identifier does not match a user or the password hash does not match, without disclosing which factor failed.
- FR-3.2.3: The system shall reject login for accounts where verified = false, prompting the user to complete registration verification first.
- FR-3.2.4: Upon valid credentials for a verified account, the system shall generate and dispatch a 6-digit OTP (purpose = LOGIN) via email and SMS, and shall respond indicating otpRequired = true without issuing a JWT.
- FR-3.2.5: The system shall provide an endpoint to submit the login OTP (step 2); upon success, the system shall establish a security context, issue a signed JWT (Bearer token), and return it together with the user's id, name, and email.
- FR-3.2.6: The system shall reject an OTP that is expired, already used, or does not match the stored hash.
- FR-3.2.7: The system shall limit OTP verification to 5 attempts per issued OTP; upon exceeding this limit, the OTP shall be invalidated and the user must request a new one.
- FR-3.2.8: OTPs (registration or login) shall expire 5 minutes after issuance.
- FR-3.2.9: The system shall provide a resend-OTP endpoint, rate-limited to one resend per 60 seconds and capped at a configurable maximum number of resends (default 3) per verification cycle.
- FR-3.2.10: The system shall provide a logout endpoint that clears the server-side security context (the client is responsible for discarding the JWT, since sessions are stateless).
- FR-3.2.11: All endpoints other than registration, login, OTP verification/resend, logout, forgot-password, and API documentation shall require a valid, unexpired JWT presented as a Bearer Authorization header.

3.3 Forgot Password / Account Recovery

Description: A three-step recovery flow designed so that a recovery attempt must be affirmatively approved by the account owner (via an emailed “Yes/No” confirmation link) before any OTP or reset capability is granted.

Functional Requirements

- FR-3.3.1: The system shall allow a user to initiate password recovery by submitting an email address or phone number identifier.
- FR-3.3.2: Upon initiation, the system shall create a recovery request with a unique token, status PENDING, and a 10-minute expiry, and shall email the user a confirmation link containing distinct “YES” and “NO” choices; if a phone number is on file, the same links shall also be sent via SMS.
- FR-3.3.3: If the user selects “NO,” the system shall mark the request status NO, take no further recovery action, and display a confirmation that the account remains unchanged.
- FR-3.3.4: If the user selects “YES,” the system shall generate a 6-digit OTP, store its hash against the recovery request (status YES), and deliver the OTP via email (and SMS if available), then display it on the confirmation page.
- FR-3.3.5: The system shall reject confirmation attempts against a token that does not exist, has already been processed (status not PENDING), or has expired (marking it EXPIRED where applicable).
- FR-3.3.6: The system shall provide a password-reset endpoint accepting the identifier, recovery token, OTP, new password, and confirmation password.
- FR-3.3.7: The system shall reject the reset if the new password and confirmation do not match, if the request status is not YES, if the request has expired, if the OTP does not match the stored hash, or if the supplied identifier does not belong to the user tied to the token.
- FR-3.3.8: Upon a successful reset, the system shall update the user's password (BCrypt-hashed) and mark the recovery request status COMPLETED.

3.4 User Profile

- FR-3.4.1: The system shall allow an authenticated user to retrieve their own profile (id, name, email, phone number, account creation timestamp).
- FR-3.4.2: The system shall return an HTTP 401 response if a profile request is made without a valid authenticated principal.

3.5 Note Management (CRUD)

Description: The core feature of the application — creating and maintaining personal notes, each optionally tied to a voice-dictated transcript.

Functional Requirements

- FR-3.5.1: The system shall allow an authenticated user to create a note with a required title (max 100 characters), required content, an optional voice transcript, an optional category (defaulting to “Others” if not supplied), and an optional color (defaulting to #ffffff if not supplied).

- FR-3.5.2: The system shall allow an authenticated user to retrieve a single note by ID, restricted to notes owned by that user.
- FR-3.5.3: The system shall allow an authenticated user to update the title, content, transcript, category, color, and pinned/archived/favorite flags of a note they own.
- FR-3.5.4: The system shall allow an authenticated user to permanently delete a note they own.
- FR-3.5.5: The system shall record createdDate at creation time and automatically update updatedDate on every modification.
- FR-3.5.6: The system shall prevent a user from reading, modifying, or deleting a note owned by a different user.

3.6 Note Organization: Pin, Archive, Favorite

- FR-3.6.1: The system shall provide an endpoint to toggle a note's pinned status and return the updated state.
- FR-3.6.2: The system shall provide an endpoint to toggle a note's archived status and return the updated state.
- FR-3.6.3: The system shall provide an endpoint to toggle a note's favorite status and return the updated state.
- FR-3.6.4: Each toggle operation shall return a contextual confirmation message reflecting the resulting state (e.g., “Note pinned successfully” vs. “Note unpinned successfully”).

3.7 Note Discovery: Search, Filter, Sort, Pagination

- FR-3.7.1: The system shall allow listing of a user's notes with optional filters for free-text search, category, pinned, archived, and favorite status, combinable in any subset.
- FR-3.7.2: The free-text search shall match against note title, content, voice transcript, and category.
- FR-3.7.3: The system shall support pagination of note listings via page and size query parameters (default size: 12).
- FR-3.7.4: The system shall support sorting of note listings by a specified field and direction (default: createdDate, descending).
- FR-3.7.5: The system shall provide dedicated convenience endpoints for retrieving pinned notes, favorite notes, archived notes, and notes filtered by a specific category.
- FR-3.7.6: The system shall provide a dedicated search endpoint accepting a free-text query parameter, returning paginated results.

3.8 Dashboard and Statistics

- FR-3.8.1: The system shall provide a dashboard endpoint returning, for the authenticated user: total note count, pinned note count, favorite note count, archived note count, a list of the user's latest notes, and a breakdown of note counts per category.

3.9 Voice Dictation (Speech-to-Text)

Description: Implemented entirely on the client using the browser's native Web Speech API; there is no backend speech-processing endpoint.

- FR-3.9.1: The system shall allow a user to dictate note content by voice directly from the browser, using the device microphone.
- FR-3.9.2: The system shall support speech recognition in English, Hindi, and Telugu, selectable by the user before or during dictation.
- FR-3.9.3: The system shall render a real-time recording/listening indicator while dictation is active.
- FR-3.9.4: The system shall allow the user to manually edit the transcribed text after dictation completes, before saving the note.
- FR-3.9.5: The system shall degrade gracefully (voice input controls disabled or hidden) when the user's browser does not support the Web Speech API, without blocking manual text entry.

3.10 Translation

Description: On-demand translation of note text between English, Hindi, and Telugu, backed by an external translation API with a local fallback.

- FR-3.10.1: The system shall provide an endpoint accepting text, a source language code, and a target language code, and shall return the translated text.
- FR-3.10.2: If the source and target languages are identical, the system shall return the original text unchanged, without an external API call.
- FR-3.10.3: The system shall normalize regional language-locale codes (e.g., en-US, hi-IN, te-IN) to their base two-letter language code before invoking the translation provider.
- FR-3.10.4: The system shall invoke the MyMemory public translation API to obtain the translated text under normal operating conditions.
- FR-3.10.5: If the external translation API is unreachable, errors, or returns an unusable response, the system shall fall back to a local, deterministic mock translation so that the endpoint still returns a response rather than failing outright.
- FR-3.10.6: The frontend shall allow a user to trigger translation of a note's content or transcript directly from the note editing/detail views.

4. External Interface Requirements

4.1 User Interfaces

The frontend is a single-page React application comprising the following primary screens, each mapped to a corresponding backend capability:

Screen	Purpose
Splash	Initial landing/branding screen shown before authentication state is resolved.
Login	Collects credentials for step 1 of two-factor login.

Screen	Purpose
Register	Collects new account details for registration.
Verify OTP	Collects the 6-digit OTP for registration verification or login step 2, with resend support.
Forgot Password	Initiates account recovery and, following email confirmation, allows OTP + new-password submission.
Dashboard	Displays note statistics, category breakdown, and recently created notes.
Notes (list)	Displays the user's notes with search, category filters, pin/favorite/archive toggles, sorting, and pagination.
Note Form	Create or edit a note, including voice dictation controls and translation trigger.
Note Details	Read-only detailed view of a single note.
Profile	Displays the authenticated user's account information.
Not Found	Fallback screen for unmatched routes.

The UI supports both light and dark themes (managed via a ThemeContext) and uses Bootstrap 5 components combined with a custom design system for cards, gradients, and status indicators (e.g., a pulsing microphone icon during active recording).

4.2 API (Software) Interfaces

The backend exposes a JSON-based REST API under the /api base path. All responses are wrapped in a common envelope of the form { success: boolean, message: string, data: <payload> }. Authenticated endpoints require an HTTP Authorization header of the form “Bearer <JWT>”. The full endpoint reference is provided in Appendix A. Interactive API documentation is auto-generated via springdoc-openapi and served at /swagger-ui.html, with the raw OpenAPI document at /api-docs.

4.3 Hardware Interfaces

- Client device microphone, accessed through the browser's Web Speech API for voice dictation. No direct hardware integration exists on the server side.

4.4 Software Interfaces

External System	Interface Type	Purpose
MySQL 8.x	JDBC (com.mysql.cj.jdbc.Driver)	Primary relational data store for users, notes, OTPs, and recovery requests.
H2 Database	JDBC (embedded)	Optional embedded database for local development/testing.
MyMemory Translation API	HTTPS REST (GET)	Provides machine translation between English, Hindi, and Telugu.
SMTP Server (e.g., Gmail)	SMTP over TLS, port 587	Delivers transactional emails: OTPs, registration confirmation, password-recovery links and codes.
Twilio	HTTPS REST	Delivers OTPs and recovery links via

External System	Interface Type	Purpose
		SMS.
Web Speech API	Browser JavaScript API	Client-side speech-to-text (and optional text-to-speech playback via a SpeechPlayer component).

4.5 Communications Interfaces

- All frontend–backend communication occurs over HTTP(S) using JSON payloads, via Axios from the React application.
- CORS is enabled on the backend for the configured frontend origins, with credentials support and an exposed Authorization header.
- The backend communicates with MyMemory and Twilio over outbound HTTPS, and with the mail server over SMTP/TLS.

5. Non-Functional Requirements

5.1 Security Requirements

- NFR-5.1.1: User passwords shall be hashed using BCrypt before storage; plain-text passwords shall never be persisted or logged.
- NFR-5.1.2: OTP codes shall be hashed using BCrypt before storage; plain-text OTPs shall exist only transiently in memory and in the outbound email/SMS message.
- NFR-5.1.3: Access to note data and profile data shall be strictly scoped to the authenticated owner; the system shall not expose one user's notes to another.
- NFR-5.1.4: Session state shall be stateless and carried via signed JWTs; the signing secret and token expiry shall be externally configurable.
- NFR-5.1.5: All configuration values that constitute secrets (database credentials, JWT signing secret, SMTP credentials, Twilio credentials) shall be externalized to application configuration and shall not be committed to source control in a production deployment; the current development configuration file stores these as plain properties and should be migrated to environment variables or a secrets manager prior to production release.
- NFR-5.1.6: Cross-origin requests shall be restricted to an explicit allow-list of frontend origins rather than a wildcard.
- NFR-5.1.7: The forgot-password flow shall require a positive, out-of-band confirmation (the emailed “YES” link) before any OTP is issued, mitigating unsolicited password-reset abuse.
- NFR-5.1.8: OTP verification and resend actions shall be rate-limited and attempt-limited as specified in Section 3.2 to reduce brute-force risk.

5.2 Performance Requirements

- NFR-5.2.1: Note listing endpoints shall return paginated results (default page size 12) to bound response payload size and query cost as note volume grows.

- NFR-5.2.2: Translation requests shall fail over to a local mock translation within the same request lifecycle if the external MyMemory API does not respond successfully, avoiding unbounded blocking on a third-party dependency.
- NFR-5.2.3: Database access for notes shall use an indexed foreign key relationship to the owning user to keep per-user queries efficient as data volume grows.

5.3 Reliability and Availability

- NFR-5.3.1: Failures in the external translation provider shall not cause the translate endpoint to return an error to the caller; a fallback response shall always be produced.
- NFR-5.3.2: A centralized global exception handler shall translate known application exceptions (validation, authentication, not-found) into structured, consistent JSON error responses rather than raw stack traces.

5.4 Usability

- NFR-5.4.1: The UI shall provide light and dark themes selectable by the user.
- NFR-5.4.2: The UI shall present clear, actionable toast notifications (via React Toastify) for the outcome of user actions (e.g., note saved, OTP resent, login failed).
- NFR-5.4.3: Voice recording state shall be visually indicated (e.g., a pulsing microphone icon) so the user always knows whether dictation is active.

5.5 Maintainability

- NFR-5.5.1: The backend shall follow a layered architecture (controller → service → repository → entity) with interfaces separating service contracts from implementations, to support testability and future extension.
- NFR-5.5.2: API documentation shall be generated automatically from code annotations (springdoc-openapi) so it stays synchronized with the implementation.
- NFR-5.5.3: Database schema evolution during development is handled via Hibernate's automatic DDL update; a migration to a versioned migration tool (e.g., Flyway or Liquibase) is recommended before production hardening.

5.6 Portability

- NFR-5.6.1: The backend shall run on any platform supporting a Java 21 runtime and shall be packaged as a self-contained executable JAR via Spring Boot's build plugin.
- NFR-5.6.2: The frontend shall build to static assets deployable to any standard static hosting or reverse-proxy environment.

6. Data Model

The persistence layer is implemented with JPA/Hibernate entities. The core entities and their key attributes are summarized below.

6.1 User

Field	Type	Constraints
id	Long	Primary key, auto-generated
name	String	Required
email	String	Required, unique, valid email format
password	String	Required, BCrypt hash
phoneNumber	String	Required, unique, max length 20
verified	boolean	Defaults false; set true after registration OTP verification
createdAt / updatedAt	LocalDateTime	Set automatically on persist/update

6.2 Note

Field	Type	Constraints
id	Long	Primary key, auto-generated
title	String	Required, max 100 characters
content	Text	Required
voiceTranscript	Text	Optional; populated from browser speech recognition
category	Category (enum)	Required; defaults to Others if not supplied
color	String	Optional; defaults to #ffffff
pinned / archived / favorite	boolean	Default false
createdDate / updatedDate	LocalDateTime	Set automatically on persist/update
user	User (FK)	Required; many-to-one, owning relationship

Category is a fixed enumeration: Work, Study, Meeting, Ideas, Shopping, Personal, Travel, Others.

6.3 Otp

Field	Type	Constraints
id	Long	Primary key, auto-generated
user	User (FK)	Required, many-to-one
otpHash	String	Required, BCrypt hash of the 6-digit code
purpose	OtpPurpose (enum)	REGISTER or LOGIN
createdAt / expiresAt	LocalDateTime	Expiry = createdAt + 5 minutes
used	boolean	Marked true once successfully verified
attempts	int	Incremented per verification attempt; capped at 5
resendCount	int	Incremented per resend; capped by app.otp.max-resends

6.4 ForgotPasswordRequest

Field	Type	Constraints
id	Long	Primary key, auto-generated
token	String	Required, unique, UUID
user	User (FK)	Required, many-to-one
status	String	PENDING → YES/NO → COMPLETED/EXPIRED
otpHash	String	Set only after “YES” confirmation
createdAt / expiresAt	LocalDateTime	Expiry = createdAt + 10 minutes

6.5 Entity Relationships

- One User has many Notes (one-to-many; a Note always belongs to exactly one User).
- One User has many Otp records over time, scoped by purpose (REGISTER / LOGIN).
- One User has many ForgotPasswordRequest records over time, one active PENDING/YES request expected at a time in normal use.

7. System Architecture

7.1 Architectural Style

The backend follows a standard layered (N-tier) architecture typical of Spring Boot applications:

- Controller layer (org.nwvs.controller): exposes REST endpoints, performs request validation via Jakarta Bean Validation, and delegates to services.
- Service layer (org.nwvs.service / service.impl): contains business logic, transaction boundaries (@Transactional), and orchestrates repositories and external integrations (email, SMS, translation).
- Repository layer (org.nwvs.repository): Spring Data JPA repositories providing data access for User, Note, Otp, and ForgotPasswordRequest.
- Security layer (org.nwvs.security): JWT generation/validation, a custom UserDetailsService, and a request filter that authenticates incoming Bearer tokens.
- DTO layer (org.nwvs.dto): request/response payload objects decoupling the public API contract from JPA entities.
- Exception layer (org.nwvs.exception): custom exceptions and a global exception handler producing consistent error responses.

The frontend follows a component-based SPA architecture using React function components, context providers for cross-cutting concerns (authentication state, theme), a dedicated Axios-based API service module, and client-side routing.

7.2 Project / Repository Structure

The repository is organized as a Maven-based Spring Boot backend at the project root, with a self-contained React/Vite frontend in a nested /frontend directory. The structure below is condensed

from the actual repository (build artifacts, IDE metadata, and node_modules are omitted for readability).

```
nwvs/                          Workspace root (Maven project)
|-- pom.xml                    Build descriptor (Spring Boot 4.1, Java 21)
|-- mvnw, mvnw.cmd             Maven wrapper scripts
|-- seed_data.sql              Reference SQL for manual data seeding
|-- voice_notes_system.postman_collection.json
|-- README.md
|
|-- src/main/java/org/nwvs/
|   |-- NwvsApplication.java   Application entry point
|   |-- config/
|   |   |-- SecurityConfig.java     JWT / CORS / filter chain
|   |   |-- OpenApiConfig.java      Swagger configuration
|   |   |-- DatabaseSeeder.java     Seeds demo users & notes
|   |-- controller/
|   |   |-- AuthController.java         Auth, OTP, recovery
|   |   |-- NoteController.java         Note CRUD, dashboard
|   |   |-- TranslationController.java  Translation endpoint
|   |-- dto/                   Request/response payloads (14 files)
|   |-- entity/
|   |   |-- User.java, Note.java, Otp.java
|   |   |-- ForgotPasswordRequest.java, OtpPurpose.java
|   |   |-- Category.java (enum)
|   |-- exception/             Custom exceptions + global handler
|   |-- repository/            JPA repositories (User, Note, Otp,
|   |                          ForgotPasswordRequest)
|   |-- security/
|   |   |-- JwtTokenProvider.java, JwtAuthenticationFilter.java
|   |   |-- CustomUserDetailsService.java, UserPrincipal.java
|   |-- service/               Business interfaces
|   |-- service/impl/          Service implementations
|   |-- util/
|       |-- OtpUtil.java       6-digit OTP generator
|
|-- src/main/resources/
|   |-- application.properties  DB, JWT, mail, Twilio, Swagger config
|   |-- static/favicon.png
|-- src/test/java/org/nwvs/
|   |-- NwvsApplicationTests.java   Spring context-load test
|
|-- frontend/                  React 19 + Vite SPA
    |-- package.json, vite.config.js
    |-- index.html
    |-- public/                favicon, icons
    |-- src/
        |-- main.jsx           Application bootstrap
        |-- App.jsx, App.css   Root component & router
        |-- components/
        |   |-- Layout.jsx         Shell layout (navbar / sidebar)
        |   |-- SpeechPlayer.jsx   Text-to-speech playback
        |-- context/
        |   |-- AuthContext.jsx    Auth state & JWT handling
        |   |-- ThemeContext.jsx   Light / dark theme state
        |-- pages/
        |   |-- Splash, Login, Register, VerifyOtp,
        |   |-- ForgotPassword, Dashboard, Notes,
        |   |-- NoteForm, NoteDetails, Profile, NotFound
        |-- services/
        |   |-- api.js             Axios API client
        |-- styles/
            |-- index.css          Design system (themes, cards)
```

7.3 Startup Data Seeding

A DatabaseSeeder component populates two demonstration users (john@example.com and jane@example.com, both with password “password123”) and a set of sample notes on first application startup, to support demonstration and manual testing without requiring manual data entry.

7.4 Deployment View

- Backend: packaged as an executable Spring Boot JAR; run via ./mvnw spring-boot:run in development or java -jar in production; listens on port 8080.
- Frontend: developed and served via Vite dev server (port 5173) during development; built to static assets (npm run build) for production hosting.
- Database: MySQL schema notes_app, auto-created if absent; connection parameters externalized in application.properties.

7.5 API Documentation and Testing Assets

- Swagger UI is available at /swagger-ui.html once the backend is running, generated automatically from controller/DTO annotations.
- A Postman collection (voice_notes_system.postman_collection.json) is included, with automatic JWT token capture from the login flow for exercising protected endpoints.

Appendix A – REST API Reference

Base path: /api. All endpoints return the common envelope { success, message, data }. Endpoints marked “Auth” require a Bearer JWT.

A.1 Authentication Endpoints (/api/auth)

Method	Path	Description	Auth
POST	/register	Register a new (unverified) account and trigger a registration OTP.	No
POST	/login	Validate credentials and trigger a login OTP (step 1 of 2FA).	No
POST	/verify-registration-otp	Verify the registration OTP and activate the account.	No
POST	/verify-login-otp	Verify the login OTP and receive a JWT (step 2 of 2FA).	No
POST	/resend-otp	Resend a new OTP (rate-limited to once per 60 seconds).	No
POST	/logout	Clear the server-side security context.	No
GET	/profile	Retrieve the authenticated user's profile.	Yes
POST	/forgot-password/initiate	Start account recovery; sends a confirmation link by email/SMS.	No
GET	/forgot-password/confirm	Callback for the emailed YES/NO confirmation link.	No

Method	Path	Description	Auth
POST	/forgot-password/reset	Verify the recovery OTP and set a new password.	No

A.2 Notes Endpoints (/api/notes)

Method	Path	Description	Auth
GET	/	List notes with optional search/category/pinned/archived/favorite filters, pagination, and sorting.	Yes
GET	/{id}	Retrieve a single note owned by the caller.	Yes
POST	/	Create a new note.	Yes
PUT	/{id}	Update an existing note.	Yes
DELETE	/{id}	Delete a note.	Yes
PUT	/pin/{id}	Toggle a note's pinned status.	Yes
PUT	/archive/{id}	Toggle a note's archived status.	Yes
PUT	/favorite/{id}	Toggle a note's favorite status.	Yes
GET	/search	Full-text search across title, content, transcript, and category.	Yes
GET	/category/{category}	Filter notes by category.	Yes
GET	/pinned	List pinned notes.	Yes
GET	/favorites	List favorite notes.	Yes
GET	/archived	List archived notes.	Yes
GET	/dashboard	Retrieve dashboard statistics and category breakdown.	Yes
POST	/translate	Translate text between English, Hindi, and Telugu.	Yes

Appendix B – Seed / Demonstration Data

Field	User 1	User 2
Name	John (sample)	Jane (sample)
Email	john@example.com	jane@example.com
Password	password123	password123
Notes	Multiple sample notes across categories, seeded on first startup	Multiple sample notes across categories, seeded on first startup

This seed data exists to support demonstration and manual QA and should be disabled or removed prior to any production deployment.

Appendix C – Assumptions, Open Items, and Recommendations

- The current application.properties file contains literal secrets (database password, JWT signing secret, SMTP credentials, Twilio credentials) checked into the project. This SRS intentionally omits reproducing those values; NFR-5.1.5 records the requirement to externalize them before production use.
- There is no distinct administrative role or content-moderation capability; all authenticated users have equal, self-scoped privileges.
- The forgot-password confirmation link is currently hardcoded to a localhost:8080 base URL, which will need to be made environment-configurable for any non-local deployment.
- No automated tests beyond a default Spring Boot context-load test were found in the codebase; a dedicated test suite (unit and integration) is recommended before production release.
- No rate limiting exists at the HTTP layer beyond the OTP-specific resend/attempt limits described in Section 3.2; general API rate limiting/throttling is recommended for a public deployment.